your `sorter` object. This sets the `end_of_data` flag ❺ and waits for the threads to finish ❻. Setting the flag terminates the loop in the thread function ⓰.

With this approach you no longer have the problem of unbounded threads that you have with a `spawn_task` that launches a new thread, and you're no longer relying on the C++ Thread Library to choose the number of threads for you, as it does with `std::async()`. Instead, you limit the number of threads to the value of `std::thread::hardware_concurrency()` in order to avoid excessive task switching. You do, however, have another potential problem: the management of these threads and the communication between them add quite a lot of complexity to the code. Also, although the threads are processing separate data elements, they all access the stack to add new chunks and to remove chunks for processing. This heavy contention can reduce performance, even if you use a lock-free (and hence nonblocking) stack, for reasons that you'll see shortly.

This approach is a specialized version of a *thread pool*—there's a set of threads that each take work to do from a list of pending work, do the work, and then go back to the list for more. Some of the potential problems with thread pools (including the contention on the work list) and ways of addressing them are covered in chapter 9.

The problems of scaling your application to multiple processors are discussed in more detail later in this chapter (see section 8.2.1).

Both dividing the data before processing begins and dividing it recursively presume that the data itself is fixed beforehand, and you're just looking at ways of dividing it. This isn't always the case; if the data is dynamically generated or is coming from external input, this approach doesn't work. In this case, it might make more sense to divide the work by task type rather than dividing based on the data.

8.1.3. Dividing work by task type